PRISMA · BEGINNER GUIDE

Transactions, Rollback & the ACID Rules

🔁 What is a transaction

⏪ What is a rollback

🧪 ACID, explained simply

Let's Start With Real Life 💸

Imagine you send 500 Taka to a friend using bKash.

Sending money is really TWO steps happening together:

Step 1: Money LEAVES your account

− 500 Taka from you

This must NOT happen alone.

Step 2: Money ARRIVES for friend

+ 500 Taka to friend

This must NOT happen alone either.

🤝 **Both steps must succeed TOGETHER, or NEITHER should happen.**
That's exactly what a database transaction guarantees.

🔁 What is a Transaction?

A transaction groups several database steps into ONE all-or-nothing unit.

📦 **All steps succeed** → changes are saved *("commit")*

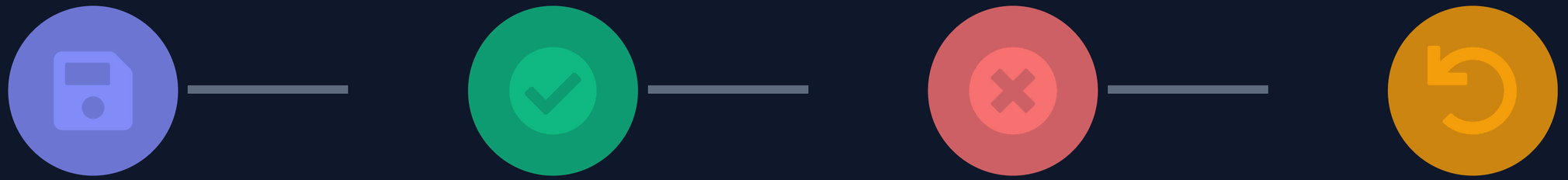
🚫 **Any step fails** → everything undone *("rollback")*

In Prisma, you write it like this 👇

```
await prisma.$transaction(async (tx) => {
  await tx.account.update({ /* -500 from sender */ })
  await tx.account.update({ /* +500 to receiver */ })
})  // ✅ both happen, or ❌ neither does
```

⏪ What is a Rollback?

A rollback is the database's "undo button." If something goes wrong mid-transaction, every change made so far is reversed — like it never happened.



Start Transaction

BEGIN

Step 1 Succeeds

Money deducted

Step 2 Fails

Receiver invalid

Rollback Triggered

Everything undone

🛡️ **Good news:** in Prisma's $transaction, rollback happens automatically — you don't write any "undo" code yourself.

📱 Real Example: bKash Refund

You send 1,000 Taka to a bKash number — but that number has NO bKash account.

1️⃣

You Send Money

1,000 Tk is deducted from your bKash balance

2️⃣

System Checks Receiver

bKash looks up the number you sent to

3️⃣

No Account Found!

The number has no bKash account — error!

4️⃣

Auto Refund (Rollback)

Your 1,000 Tk is put back. As if nothing happened

✅ **This refund IS a rollback — the deduction (step 1) gets undone because step 3 failed.**

💻 Same Story, in Prisma Code

This is how a bKash-style transfer might look using Prisma's interactive transaction:

```
await prisma.$transaction(async (tx) => {
  // 1️⃣  Deduct money from sender
  const sender = await tx.account.update({
    where: { number: senderNumber },
    data:  { balance: { decrement: 1000 } }
  })
  // 2️⃣  Check the receiver exists
  const receiver = await tx.account.findUnique({
    where: { number: receiverNumber }
  })
  // 3️⃣  No account? Throw — Prisma rolls everything back
  if (!receiver) throw new Error("No bKash account!")
})
```

Meet ACID

ACID is the set of 4 promises every reliable database transaction makes. It's the reason your bKash balance never just disappears.

A

Atomic

C

Consistent

I

Isolated

D

Durable

A Atomic

"All or nothing"

Every step in a transaction is treated as ONE single, indivisible unit. There's no "half-done" state.

❌ Without Atomicity

– 1,000 Tk leaves your account...
but the app crashes before it
reaches the receiver.

Result: money just vanishes! 😱

✅ With Atomicity

App crashes mid-transfer?
Prisma rolls back the deduction too.

Result: your 1,000 Tk is
still safe in your account. 🛡️

C Consistent

"The rules are always obeyed"

A transaction can never leave the database in a broken or illogical state — the total of all money must always add up.

Before the transfer:

You: 5,000 Tk + Friend: 2,000 Tk = Total: 7,000 Tk

After the transfer:

You: 4,000 Tk + Friend: 3,000 Tk = Total: 7,000 Tk

📊 **The total Taka in the system (7,000) never changes — money only moves, it never appears or disappears.**

I Isolated

"One transaction at a time, please"

Two transactions running at the same time can't see each other's half-finished work — they act as if running alone.

👩 You: Send 500 Tk

Reading your balance...

Deducting 500 Tk...

Saving new balance...

🔒 *Locked — others must wait their turn for this account*

🧑 Friend: Checks Balance

Sees OLD balance until your transaction fully finishes.

No confusing in-between number.

✨ *Sees a clean before-or-after state, never a messy middle*

D Durable

"Saved means saved — forever"

Once a transaction is committed, it survives — even if the power goes out or the server crashes a second later.

Transfer Confirmed

bKash shows "Transaction Successful"

Saved to Disk

Committed permanently to the database

Phone Dies / App Closes

Doesn't matter — it's already saved

Reopen App Later

Balance is still correctly updated

📋 ACID — Quick Recap

A	**Atomic**	All steps happen, or none do	📦
C	**Consistent**	Money is never created or lost	⚖️
I	**Isolated**	Transactions don't interfere with each other	🔒
D	**Durable**	Once saved, it's saved for good	💾

Now you know why your money is always safe. 🛡️

🔁 Transaction = all-or-nothing

⏪ Rollback = the undo button

🧪 ACID = the 4 safety rules